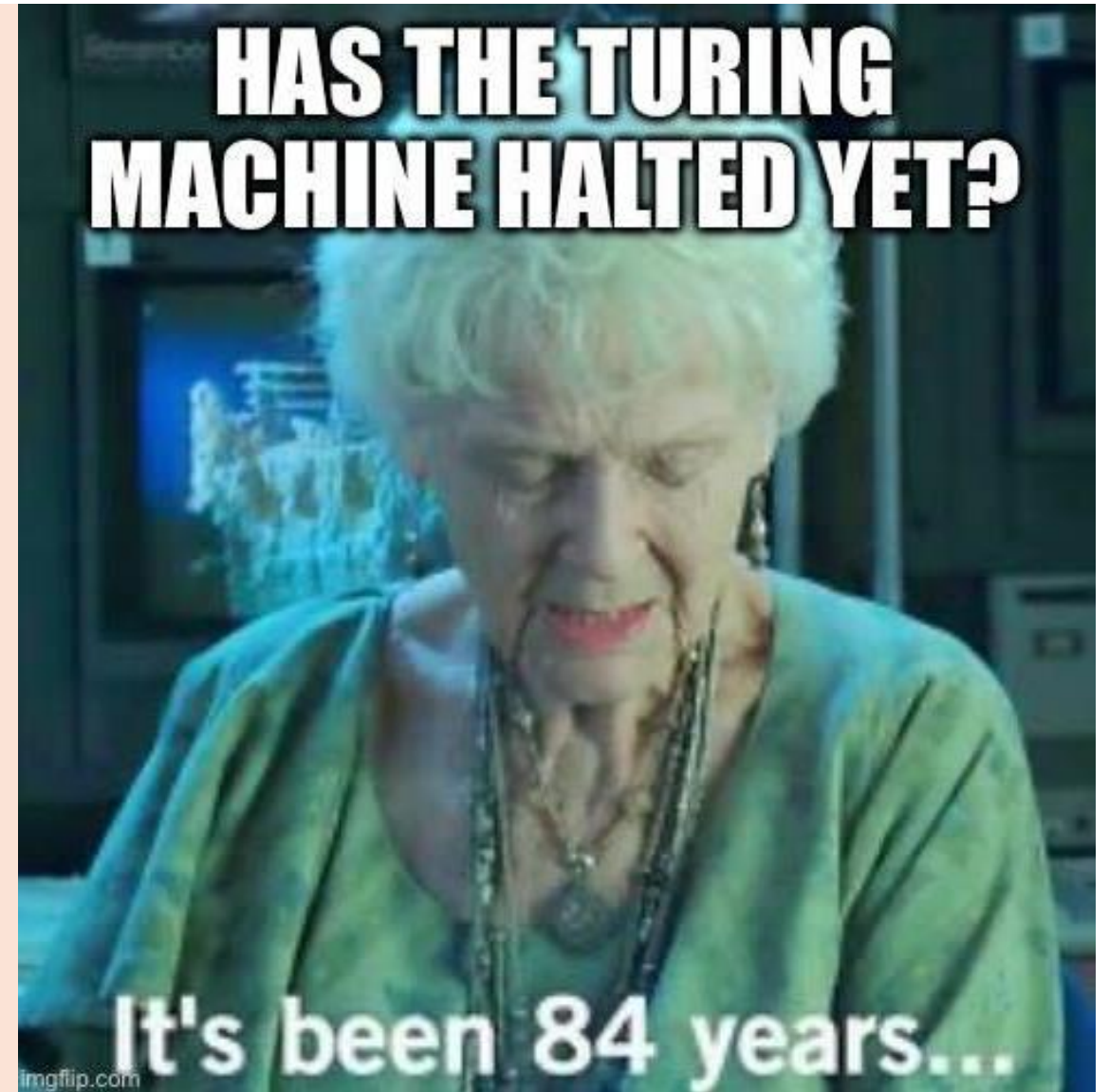


COMP30026

Models of Computation

Cezary Kaliszyk and **William Umboh**

Lecture 20 – Undecidability



Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Announcements

- Office hours:
 - Cezary: Thursdays at 10 am till end of semester (Lectures 1 – 15)
 - William: Thursdays at 3 pm in Melbourne Connect Open Area 2209 (Lectures 16 onwards) Free coffee!
- Make use of student hub!
- Assignment 2
 - Out now; **Due: W12 Monday 10 am**
 - Solutions and feedback released Monday W13

Overview

- Recall: Church-Turing Thesis says

Computable = Decidable

UnComputable = UnDecidable

- Today: Show that there are undecidable languages
 1. There is an unrecognizable language (💡 **universality** and **code = data**)
 2. TM Acceptance is undecidable (💡 **reduction**)
 3. 2 proofs (diagonalization and self-reference)
- Takeaway: There are computational problems that cannot be solved
 - There is no algorithm that on **every** input, **correctly** output yes/no in **finite time** (i.e. does not loop)

TM Acceptance (A_{TM})

- First example of a recognizable language
- Given encoding of a TM M and input w , decide if M accepts w
 - Equiv: Given source code of Python program and w , does it output 1 on w ?
- Let $A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$

TM Acceptance (A_{TM})

Thm. $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is recognizable.

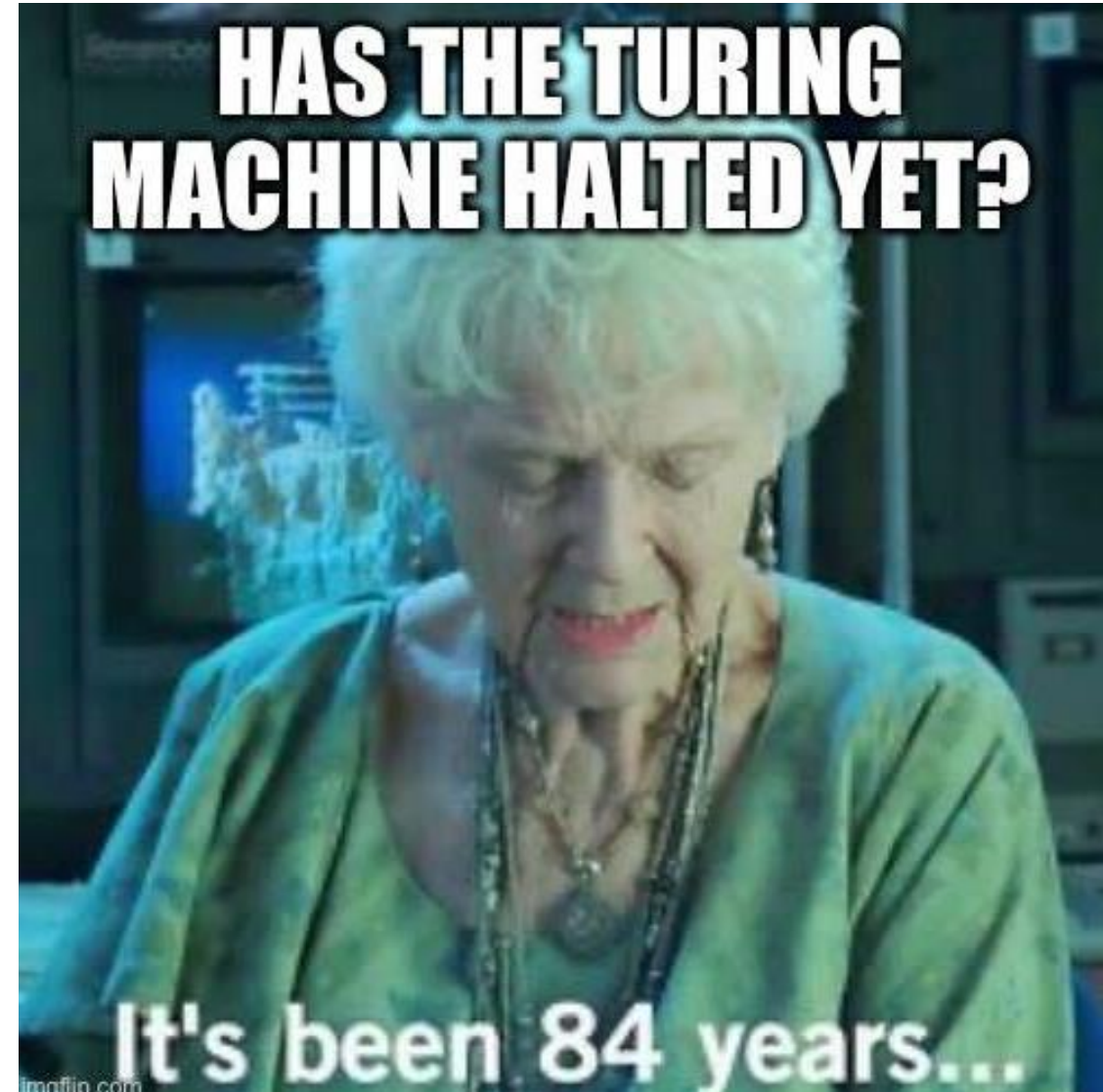
Pf. Consider the following TM

U = “On input $\langle M, w \rangle$,

1. Simulate M on w
2. If M accepts, accept; else reject”

✗ Not a valid TM!

Line 2: What if M does not halt?



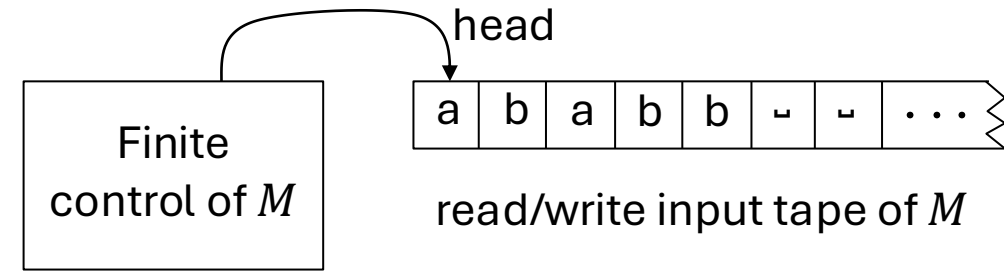
TM Acceptance (A_{TM})

Thm. $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is recognizable.

Pf. Consider the following TM

U = “On input $\langle M, w \rangle$,

1. Let C be initial configuration of M on w
2. While current configuration C is neither accept nor reject
 - Update C to be next configuration using transition function of M
3. If C is accept, accept; else reject”



Correctness: U accepts if and only if M accepts w

- Case 1 (M accepts w): U accepts
- Case 2 (M rejects w): U does not accept
- Case 3 (M does not halt): U does not accept

QED



Idea: Universality (Code = Data)

U = “On input $\langle M, w \rangle$,

1. Let C be initial configuration of M on w
2. While current configuration C is neither accept nor reject
 - Update C to be next configuration using transition function of M
3. If C is accept, accept; else reject”

- U is a **universal** TM: it can simulate all other “special-purpose” TMs
- Revolutionary idea!
 - Stored-program computer ($\langle M \rangle$ = code; $\langle w \rangle$ = data/input to run code on)
 - Without this idea: need one device for TikTok, another for Instagram, etc.
- Example: Python interpreter on Ed (this is how your code is evaluated on test cases)

A_{TM} is undecidable: Roadmap

1. Construct an unrecognizeable language L_D
 - Techniques: Diagonalization (Cantor) + Universality (Code = Data)
2. Show that if A_{TM} is decidable, then L_D is decidable
 - Technique: Reduction
 - Reduce deciding L_D to deciding A_{TM}
3. (1) and (2) implies that A_{TM} is undecidable

1. Construction of L_D

- Let $\langle M \rangle$ be a **binary string** encoding of TM M
 - Can think of $\langle M \rangle$ as the bits of Python source for an algorithm M
- Enumerate all TMs in an infinite table
 - Let M_i be i -th TM in string/lexicographic order
 - Let w_j be j -th binary string in string/lexicographic order:
 - $w_0 = \epsilon, w_1 = 0, w_2 = 1, w_3 = 01, w_4 = 11, \dots$
- Every TM M appears in some row and every string in some column

	$w_0 = \epsilon$	$w_1 = 0$	$w_2 = 1$	$w_3 = 01$	...
M_0	1	1	1	1	...
M_1	1	0	1	0	...
M_2	0	0	1	0	...
M_3	0	1	1	1	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table: 1 in cell (i, j) means M_i accepts w_j

1. Construction of L_D

- Define L_D by flipping **diagonal** of table

Thm. L_D is not recognizable.

Pf. For every i :

- If $w_i \in L_D$, then TM M_i does not accept w_i
- If $w_i \notin L_D$, then TM M_i accepts w_i
- Thus, M_i makes a mistake on w_i

Every TM M is M_i for some i

Thus, no TM recognizes L_D

QED

$w_i \in L_D$ iff M_i does not accept w_i

	$w_0 = \epsilon$	$w_1 = 0$	$w_2 = 1$	$w_3 = 01$	...
L_D	0	1	0	0	...

	$w_0 = \epsilon$	$w_1 = 0$	$w_2 = 1$	$w_3 = 01$	...
M_0	1	1	1	1	...
M_1	1	0	1	0	...
M_2	0	0	1	0	...
M_3	0	1	1	1	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table: 1 in cell (i, j) means M_i accepts w_j

2. If A_{TM} decidable, then L_D decidable

Pf. Suppose M decides A_{TM}

Construct D to decide L_D

Idea: Use M to look up table

D = “On input w :

1. Compute i such that $w = w_i$
2. Simulate M on input $\langle M_i, w_i \rangle$
3. If M rejects (i.e., M_i does not accept w_i), accept; else reject”

Since M decides A_{TM} , D correctly decides L_D

QED

$w_i \in L_D$ iff M_i does not accept w_i

	$w_0 = \epsilon$	$w_1 = 0$	$w_2 = 1$	$w_3 = 01$	...
L_D	0	1	0	0	...

	$w_0 = \epsilon$	$w_1 = 0$	$w_2 = 1$	$w_3 = 01$	...
M_0	1	1	1	1	...
M_1	1	0	1	0	...
M_2	0	0	1	0	...
M_3	0	1	1	1	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table: 1 in cell (i, j) means M_i accepts w_j

A_{TM} is undecidable: Roadmap

1. Construct an unrecognizable language L_D
 - Techniques: Diagonalization (Cantor) + Universality (Code = Data)
2. Show that if A_{TM} is decidable, then L_D is decidable
 - Technique: Reduction
 - Reduce recognizing L_D to deciding A_{TM}
3. (1) and (2) implies that A_{TM} is undecidable

Self-Reference

- Logic: “I am a liar” (liar = every statement false)
- Computing: relies on universality to run a TM M on its own code $\langle M \rangle$
- Next: use self-reference and proof by contradiction to prove unrecognizability and undecidability

Construction of L_S via Self-Reference

L_S consists of encodings $\langle M \rangle$ of TMs M
s.t. M does not accept $\langle M \rangle$

Note: If string w is not a valid encoding of
a TM, then $w \notin L_S$

Thm. L_S is not recognizable.

Pf. Suppose, towards a contradiction,
that TM R recognizes L_S

Since R recognizes L_S , R accepts $\langle R \rangle$ iff
 $\langle R \rangle \in L_S$

By definition of L_S , $\langle R \rangle \in L_S$ iff R rejects
 $\langle R \rangle$, contradiction!

QED

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
L_S	0	1	0	0	...

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
M_0	1	1	1	1	...
M_1	1	0	1	0	...
M_2	0	0	1	0	...
M_3	0	1	1	1	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table: 1 in cell (i, j) means M_i accepts $\langle M_j \rangle$

Undecidability of A_{TM} via Self-Reference

Suppose, towards a contradiction, that some TM H decides A_{TM}

Use H to construct a “self-contradicting” machine D

D = “On input $\langle M \rangle$:

1. Simulate H on $\langle M, \langle M \rangle \rangle$
2. Accept if H rejects; reject if H accepts”

What happens when we run D on $\langle D \rangle$?

D accepts $\langle D \rangle$ iff H rejects $\langle D, \langle D \rangle \rangle$

By definition of A_{TM} , H rejects $\langle D, \langle D \rangle \rangle$ iff D does not accept $\langle D \rangle$

Contradiction! QED

Summary

- Recall: Church-Turing Thesis says

Computable = Decidable

UnComputable = UnDecidable

- Today: Show that there are undecidable languages
 1. There is an unrecognizable language (💡 **universality** and **code = data**)
 2. TM Acceptance is undecidable (💡 **reduction**)
 3. 2 proofs (diagonalization and self-reference)
- Takeaway: There are computational problems that cannot be solved
 - There is no algorithm that on **every** input, **correctly** output yes/no in **finite time** (i.e. does not loop)